R Software

R

R is a powerful programming language widely used for statistical computing and graphics. It provides a variety of statistical techniques and graphical tools, making it a useful tool for data analysis and visualization.

R with Conda

Conda is a package manager commonly used for Python, but is compatible with R and can be very useful for installing packages. This can be helpful when the packages you need have specific dependency requirements. Because of these benefits, **Conda is our recommended process for using R on the cluster.** You can find more information on Conda in the Python software section.

When you create a Conda environment, you can specify exactly the packages you need. Conda is available through the Minforge module:

```
module load miniforge/24.3.0-0
```

Now, you should be able to run `conda` commands, such as creating and activating an environment:

```
conda create -n my_R_environment
conda activate my_R_environment
```

To search for specific R packages (beginning with "r-"), you can use `conda search`. For example, the following looks for all versions of Tidyverse available through Conda:

```
conda search r-tidyverse
```

The base R installation through Conda is called `r-base`. This will automatically be downloaded and installed when you install any R package to your environment. To install packages, use `conda install`:

```
conda install r-tidyverse
```

By default, the latest compatible versions of `r-base` and other R packages are automatically installed. If you prefer different versions of R (`r-base`) or you need a specific version of a package, you can specify in your `install` command:

```
conda install r-base=4.1.2
conda install r-tidyverse=2.0.0
```

Note

It's much more efficient to specify all the packages you need when you first create your environment rather than installing them one by one. This way, the environment only needs to be solved once, and Conda ensures that packages are compatible with each other. You can do this by naming the packages in the `create` command: `conda create -n my_R_env r-tidyverse r-pillar`

Once your environment is created and activated, entering `which R` should direct you to the version of R within your Conda environment.

Pre-Installed R Modules

There are currently a few different versions of R installed on Engaging. You can find these versions by running `module avail`. To use an R interactive environment, first load an R module, then enter `R`.

```
module load r/4.2.2-x86_64
R
```

Installing Packages

The pre-installed R modules come with a limited number of packages, but it is possible to install more. This can be acheived using the standard R command `install.packages("packageName")`.

R will first try to install this package system-wide but will be blocked to avoid editing the module for the entire cluster. You will be asked if you want to use a personal library instead. Type "yes" and press enter. This creates a folder in your home directory that contains the packages for each version of R you use. You can check this directory by running `.libPaths()` from the R CLI.

RStudio

You can use RStudio on the Engaging cluster via ORCD OnDemand > Interactive Apps > RStudio Server. From there, select the specifications you need, including runtime, memory, and R version.

Using the RStudio Module on Engaging

Currently, OnDemand does not support local installations of R or versions of R installed through Conda. This can make it difficult to install certain packages. While we plan on updating this soon, we have a current workaround that takes a bit of setup but makes packages much easier to install.

First, you will need to start a job on Engaging. This can be an interactive job or a batch job, but we will use an interactive job in the current example:

```
salloc -N 1 -n 4 --mem-per-cpu=4G -p mit_normal
```

Be sure to edit the flags as necessary to request the resources you need, such as CPU cores, memory, or GPUs. See Requesting Resources for more information.

Next, load the `rstudio/4.4.2` module and run the `rstudio` command:

```
module load rstudio/4.4.2
rstudio
```

You should now see printed instructions for setting up port forwarding to interact with RStudio using your web browser. If using a batch session, these instructions will appear in your job output file.

Jupyter

Similar to RStudio, Jupyter notebooks offer a handy cell-based interface to run R code. You can run R on Jupyter notebooks through the Engaging web portal.

Jupyter notebooks are available through ORCD OnDemand > Interactive Apps > Jupyter Notebook. To run R, you must create a Conda environment with both `r-irkernel` and `jupyterlab` installed (see R with Conda and Jupyter). When starting up the notebook, enter the name of your custom Conda environment. Once you launch the session and open your notebook, you may need to change your kernel to R. Your current kernel is shown in the top right, and likely defaults to "Python 3 (ipykernel)". Click this to change it to R.

FAQs

I am trying to use a specific R installation, but it is not being recognized. What should I do?

Sometimes, the way your environment is set up may cause the system to default to certain R installations that you don't want. The culprit can often be found in your `.bashrc` and/or `.bash_profile` file. Usually, running `module purge` from the command line before loading the version of R you want solves the problem.

How do I change the path where my libraries are installed?

Before starting R, you can set the `R_LIBS_USER` environment variable from your Bash terminal:

```
export R_LIBS_USER=/path/to/R/library/directory
```

You can also set the path from within R:

```
.libPaths("/path/to/R/library/directory")
```

Both of these commands essentially prepend your custom path to the library path that already existed.

April 29, 2026